```
  status: string,
  rowNumber: number,
  seatNumber: number
): VenueThunk => {
  return async (dispatch, getState) => {
    const actionType = status === "unsold" ? "holdTicket" : "unholdTicket"
    await subscription.perform("added_to_cart", {
      concertId: getState().concertId,
      row: rowNumber,
      seatNumber: seatNumber,
      status: actionType === "holdTicket" ? "held" : "unsold",
      ticketsToBuyCount: getState().ticketsToBuyCount,
    })
    dispatch({ type: actionType, seatNumber, rowNumber })
  }
}

export const clearCart = (): VenueThunk => {
  return async (dispatch, getState) => {
    await subscription.perform("removed_from_cart", {
      concertId: getState().concertId,
      tickets: getState().myTickets,
    })
    dispatch({ type: "clearHolds" })
  }
}
```

The pattern is similar: we have an outer function that returns a VenueThunk and an inner function that uses the dispatch and state to do the real work. Again, we're declaring the inner function as async and using await on the subscription call. My IDE warns that it does not think the await is needed on the subscription.perform call, which I suspect is an issue in the ActionCable type settings.

With that in place, here's the entire Subtotal component using Redux hooks and Redux Thunk:

chapter_12/02/app/javascript/components/subtotal.tsx

```
import * as React from "react"
import styled from "styled-components"
import {
  clearCart,
```